CHƯƠNG 4:
KẾ THỪA (INHERITANCE)

KẾ THỪA

- Kế thừa là một trong các đặc điểm chính của Lập trình hướng đối tượng.
- Khi ta cần đặc biệt hoá đối tượng đã có: hãy sử dụng kế thừa bằng cách thêm một số thuộc tính và phương thức cần thiết.
- Kế thừa giúp người lập trình tiết kiệm được thời gian và công sức vì không phải xây dựng lại từ đầu mọi thứ đã có, sử dụng lại phần code đã viết.

VÍ DỤ

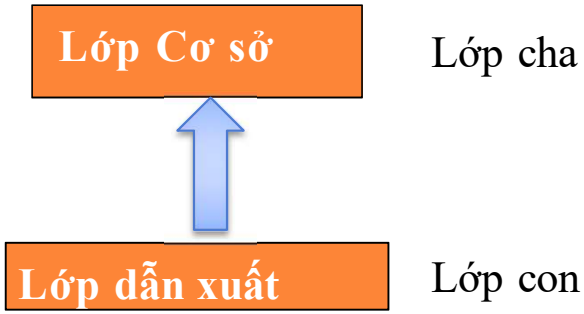
- Ta đã xây dựng class **Nguoi** bao gồm các thuộc tính: họ tên, địa chỉ,.., các phương thức nhập, xuất, hiệu chỉnh...
- Ta có thể xây dựng lớp **SinhViên** thừa kế từ lớp Người bằng cách thêm các thuộc tính: MaSV, lớp và phương thức nhập (chỉ cần viết thêm phần nhập MaSV, lớp), xuất (chỉ cần viết thêm phần xuất MaSV, lớp,...)

VÍ DỤ

- Ta đã xây dựng lớp **Điểm** bao gồm các thuộc tính x,y và phương thức nhập, xuất, di chuyển,..
- Ta có thể xây dựng lớp điểm có màu kế thừa từ lớp điểm bằng cách bổ sung thuộc tính màu,...
- Ví dụ: Chó kế thừa từ lớp Động vật
- Ví dụ: Trưởng phòng kế thừa từ lớp Nhân viên

LỚP CƠ SỞ - LỚP DẪN XUẤT

- Lớp cơ sở (base class): là lớp đã được xây dựng
- Lớp dẫn xuất (derived class): là lớp cần được xây dựng dựa trên lớp cơ sở



Lớp cha

Lớp con

KHAI BÁO

Cú pháp:

[thuộc tính truy xuất] class <tên lớp dẫn xuất> extends <tên lớp cơ sở>

```
{
   // các thuộc tính bổ sung
   // các phương thức bổ sung
      // các phương thức mới trùng tên với phương thức của lớp cơ sở
}
```

Từ khóa Super

Từ khóa **Super:**

- super được sử dụng để truy cập đến các thành viên của lớp cha gần nhất.
- super() được sử dụng để gọi constructor của lớp cha gần nhất.

VÍ DỤ

```
class Nguoi
{
   String ho;
   String ten;
   public Nguoi(){ }
   public Nguoi(String h,String t) {
         ho=h;  ten=t;
   }
   public Nguoi(Nguoi ng1) {
         ho=ng1.ho;     ten=ng1.ten;
   }
   public void xuat() {
         System.out.print("Ho va ten: " + ho + " " + ten);
   }
}
```

VÍ DỤ (TT)

```
class SinhVien extends Nguoi
{
   String masv;
   public SinhVien (String h, String t, String m)
   {
         super(h,t);
         masv=m;
   }
   public void xuat()
   {
         super.xuat();
         System.out.print(" Ma SV: "+masv);
   }
}
```

Gọi hàm thiết lập 2 tham số của lớp Nguoi

Phương thức trùng tên với phương thức của lớp cơ sở

Gọi hàm xuat() của lớp Nguoi

VÍ DỤ (TT)

```
class SinhVienTest {
   public static void main(String[] args)
   {
         Nguoi ng1 = new Nguoi(“Nguyen Van”, “Tieu”);
         ng1.xuat();
         SinhVien sv1 = new SinhVien(“Tran Van”, “Tung”, “310712222”);
         sv1.xuat();
   }
}
```

TRUY XUẤT

- Lớp con được phép truy xuất các thành phần protected và public của lớp cha.
- Lớp Object là lớp gốc, tất cả các lớp đều kế thừa từ lớp Object.

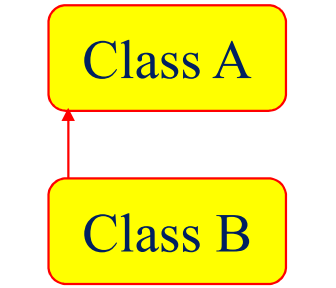
```
public class DONGVAT
{
        String ten;
        float   cannang;
        public DONGVAT()
        {
                ten="DV";
                cannang=0;
        }
        public DONGVAT(String t,float cn)
        {
                this.ten=t;
                this.cannang=cn;
        }
        public void keu()
        {
                System.out.print("\n AAA!!!");
        }
        public void xuat()
        {
                System.out.print("\nTen: "+ten+"\nCan nang: "+cannang+" Kg");
        }
}
```

```
class MEO extends DONGVAT
{
        String maulong;
        public MEO()
        {
                maulong = "";
        }
        public MEO (String ten, float cannang, String maulong)
        {
                super(ten, cannang);
                this.maulong = maulong;
        }
        public void keu()
        {
                System.out.print("\n MEO MEO!!!");
        }
        public void xuat()
        {
                super.xuat();
                System.out.print("\n mau long: " + maulong);
        }
}
```

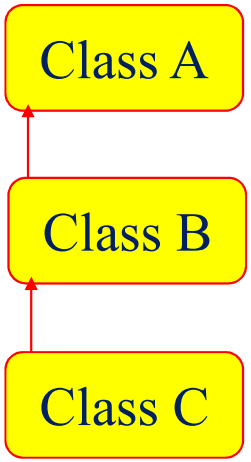
Mặc định sẽ gọi hàm thiết lập không tham số của lớp cha

KẾ THỪA

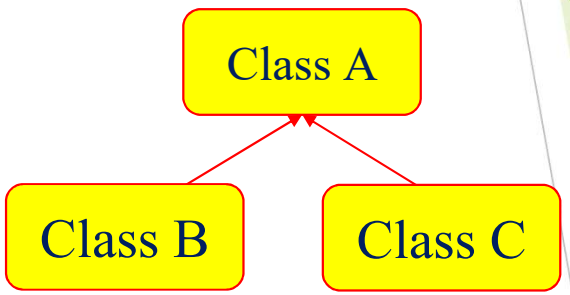
Có 3 kiểu kế thừa: *đơn kế thừa, kế thừa nhiều cấp, kế thừa thứ bậc*.



Đơn kế thừa



Kế thừa nhiều cấp



Kế thừa thứ bậc

SỰ CHUYỂN KIỂU (CASTING)

- Chuyển kiểu **ngầm định (implicit)**
- Chuyển kiểu **tường minh (explicit)**

Ví dụ:

```
int i = 100;
long l1= i;          //implict
long l2 = (long) i; //explicit
```

- **Upcasting:** Chuyển đối tượng là thể hiện của lớp con thành đối tượng là thể hiện của lớp cha (ngầm định hoặc tường minh)
- **Downcasting:** Chuyển đối tượng là thể hiện của lớp cha thành đối tượng là thể hiện của lớp con (tường minh)

```
class DongVatTest {
    public static void main(String[] args) {
        DONGVAT dv1 = new DONGVAT();
        dv1.xuat();
        dv1.keu();
        MEO m1 = new MEO("mimi", 3, muop");
        m1.xuat();
        m1.keu();
        MEO m2 = new MEO("binbin", 2, "mun");
        m2.xuat();
        m2.keu();
        dv1=m1;
        dv1.xuat();
        dv1.keu();
        m2=(MEO)dv1;
        m2.xuat();
        m2.keu();
    }
}
```

Dong vat co ten la: DV va can nang la: 0.0 Tieng keu la AA

Dong vat co ten la: mimi va can nang la: 3.0
Mau long la: muop Tieng keu la: Meo meo

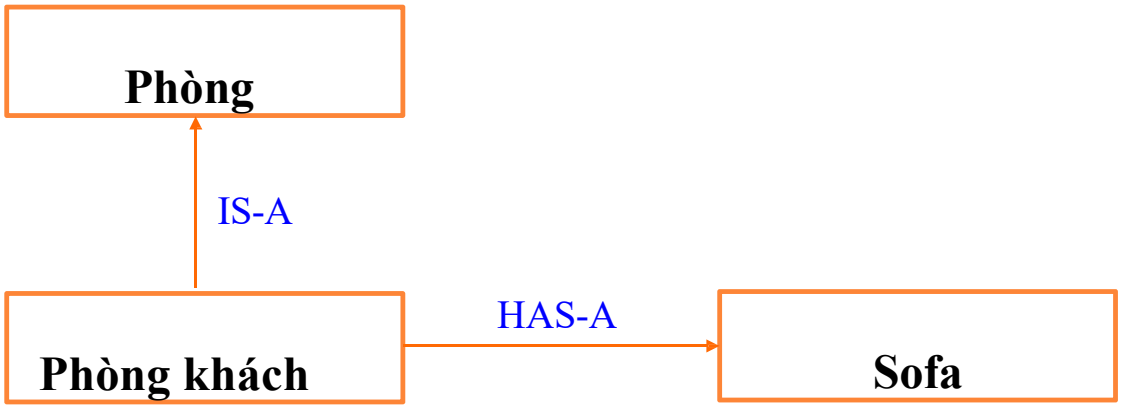
Dong vat co ten la: binbin va can nang la:
2.0 Mau long la: mun Tieng keu la: Meo meo

Dong vat co ten la: mimi va can nang la: 3.0
Mau long la: muop Tieng keu la: Meo meo

Dong vat co ten la: mimi va can nang la: 3.0
Mau long la: muop Tieng keu la: Meo meo

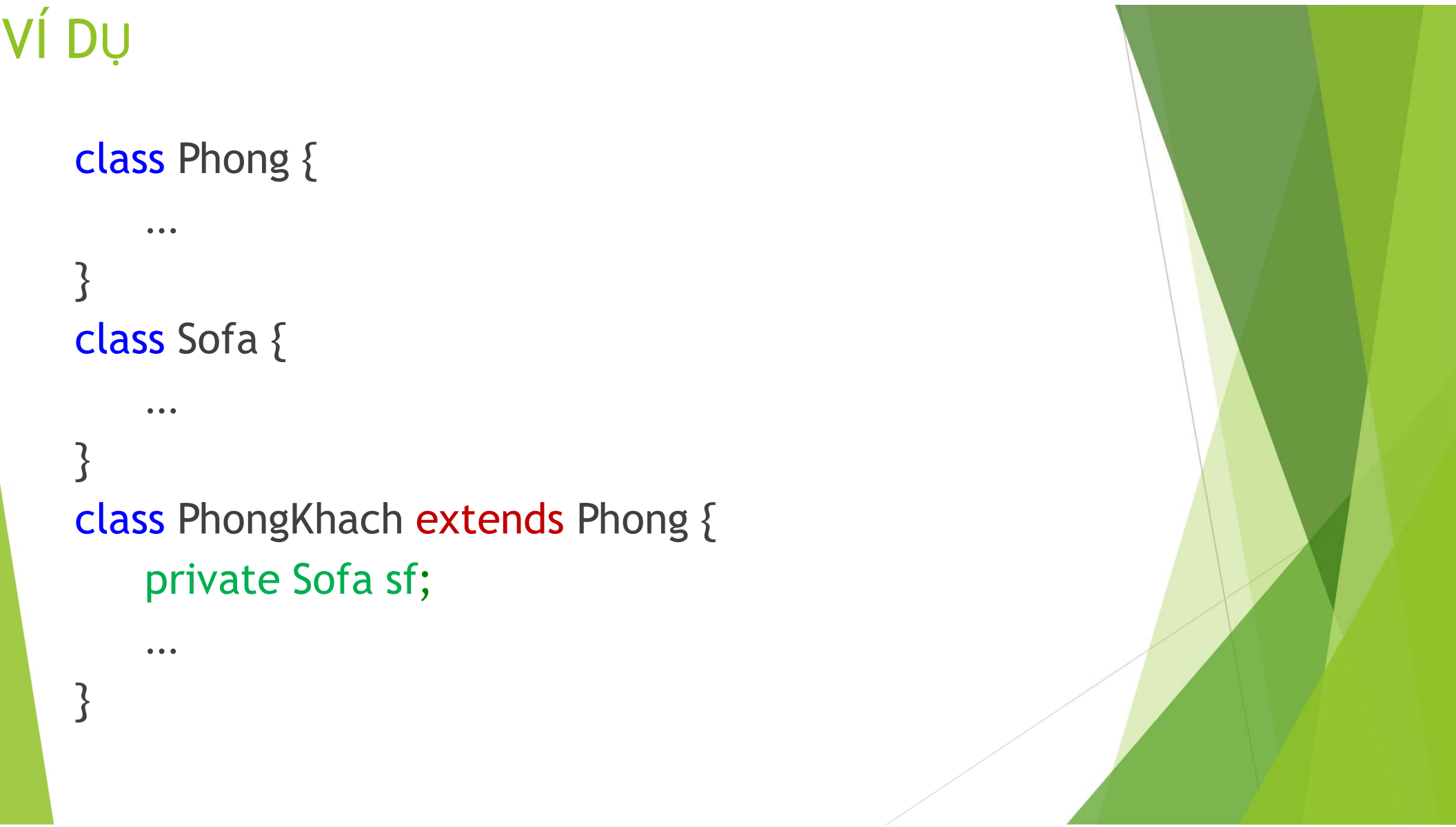
QUAN HỆ IS-A VÀ HAS-A

- **IS-A** là quan hệ kế thừa (inheritance).
- **HAS-A** là quan hệ hợp thành (composition), hay nói cách khác mô tả đối tượng có thành viên là một đối tượng của lớp khác.
- **Ví dụ:**



VÍ DỤ

```
class Phong {
    …
}
class Sofa {
    …
}
class PhongKhach extends Phong {
    private Sofa sf;
    …
}
```



BÀI TẬP 1

- Thiết kế lớp NoiSinh với thành phần dữ liệu phường, quận, tỉnh thành.
- Viết các phương thức khởi tạo, các phương thức get/set, phương thức nhập/xuất cho lớp NoiSinh.
- Thiết kế lớp Nguoi gồm số định danh, Họ tên, năm sinh, địa chỉ và nơi sinh.
- Viết các phương thức khởi tạo, phương thức get/set, phương thức nhập/xuất cho lớp Nguoi.
- Thiết kế lớp SinhVien kế thừa từ lớp Nguoi, bổ sung thêm thuộc tính MASV (int), lớp (String).
- Viết các phương thức khởi tạo, phương thức get/set, phương thức nhập/xuất cho lớp SinhVien.

BÀI TẬP 2

- Thiết kế lớp điểm gồm 2 thuộc tính x, y (int) và các phương thức thiết lập/nhập/ xuất.

- Thiết kế lớp điểm màu thừa kế từ lớp điểm, bổ sung thuộc tính màu (int), các phương thức thiết lập/nhập/xuất của lớp điểm màu.

- Thiết kế lớp đoạn thẳng bao gồm các thuộc tính điểm màu 1, điểm màu 2 (là đối tượng của lớp điểm màu), các phương thức:

+ Thiết lập/nhập/xuất

+ Tính khoảng cách giữa 2 điểm

+ In ra kết quả cho biết 2 điểm của đoạn thẳng có cùng màu không

- Viết hàm main để test: tạo 1 đối tượng đoạn thẳng và gọi thực thi các phương thức của nó

BÀI TẬP 3

- Sử dụng lớp điểm đã tạo ở Bài tập 2.
- Thiết kế lớp tam giác với thành phần dữ liệu gồm 3 đỉnh là 3 điểm.
- Viết các phương thức khởi tạo, phương thức get/set, phương thức nhập/xuất cho lớp tam giác, và phương thức tính chu vi của tam giác.
- Viết hàm main để test: tạo 1 đối tượng hình tam giác và gọi thực thi các phương thức của nó.